

Laboratoire de Programmation Temps Réel Semestre printemps 2025 - 2026

Laboratoire 3: Linux et Xenomai EVL

Temps à disposition: 4 périodes

Laboratoire en groupe de 2 personnes

Objectifs

Ce laboratoire va vous permettre d'expérimenter les différences de performances entre:

1. Votre programme `signal_timer2.c` basé sur un timer périodique `CLOCK_REALTIME` du laboratoire 2 précédent.
2. Un thread posix utilisant le scheduler Linux `SCHED_FIFO` avec une haute priorité (similaire au développement du programme `priority.c` du laboratoire 1 précédent).
3. Un thread temps réel Xenomai EVL.

Préconditions au laboratoire 3

Afin d'augmenter le déterminisme des exécutions des programmes et de supporter l'analyse, sauf indication contraire, tous vos programmes devront être exécutés sur un seul CPU. Cela peut être réalisé soit grâce à la commande `taskset` ou directement au niveau des programmes, avec les fonctions `CPU_ZERO`, `CPU_SET` et `sched_setaffinity`.

Étape 1: `signal_timer2.c` - Mesures de performance

Lors du dernier labo, vous avez testé les capacités de votre application `signal_timer2.c` à servir des tâches périodiques sous différentes conditions. Nous allons maintenant adapter ce programme afin de mesurer ces performances à l'aide d'un GPIO.

Étudiez et exécutez le code fourni `./base_app/gpio_toggle/gpio_toggle.c`. Pour arrêter le programme, effectuer un `ctrl-C`.

```
$ ./gpio_toggle
```

A l'aide des documents de la DE1-SoC fournis sur cyberlearn ...

- `DE1-SoC_User_manual.pdf`
- `DE1-SoC_schematic.pdf`

- ✓ ... pouvez-vous expliquer comment le programme `gpio_toggle.c` fonctionne? Afin de valider votre compréhension, veuillez connecter un oscilloscope afin d'observer la fonction du programme. Analysez les écarts observés sur le scope.
- ✂ Modifiez votre programme `./base_app/signal_timer2/signal_timer2.c` du précédent laboratoire pour qu'il puisse mesurer la régularité de l'exécution d'une tâche périodique en utilisant un GPIO. Vous devrez effectuer cette mesure sur des intervalles de 10ms.
- ✓ Ensuite, comparez les résultats des durées mesurées entre deux occurrences entre (1) des mesures effectuées avec un oscilloscope et (2) les mesures reportées par votre programme (à l'aide de la fonction `clock_gettime()`). Analysez les écarts entre ces deux méthodes de mesure et discutez des résultats.

Étape 2: Environnement chargé

Nous allons maintenant effectuer des tests et récolter des données dans un environnement plus chargé en termes d'utilisation du temps processeur. Afin de créer un tel environnement, nous allons utiliser les commandes suivantes:

```
# taskset -pc 0 $$
# stress --cpu 8 --io 4 --vm 1 --timeout 10s
```

Utilisez les pages man ou l'aide de `stress` afin de comprendre ce que fait chaque option

- ✓ Collectez les statistiques en exécutant l'application `signal_timer2.c` exécuté depuis un autre shell et comparez vos résultats avec les résultats précédent.
- N'oubliez pas d'exécuter la même commande `taskset` si vous utilisez deux terminaux.

Étape 3: Tâches périodiques à haute priorité sous Linux

Avec notre environnement prêt pour réaliser les expérimentations, il est temps de l'exploiter avec des approches distinctes.

- ✂ Développez un programme nommé `./base_app/pthread_timer/pthread_timer.c`. Ce programme doit lancer une tâche Linux périodique fonctionnant sous une politique d'ordonnancement `SCHED_FIFO` avec à une priorité élevée. Utilisez spécifiquement la fonction `nanosleep()` afin de gérer des périodes d'attente de 10ms à l'intérieur d'une boucle, permettant ainsi de contrôler avec précision les intervalles entre chaque exécution de la tâche.
- ✓ A nouveau, comparez les résultats des durées mesurées entre deux occurrences. Pour ce faire, utilisez les mêmes approches pour caractériser les performances du programme (GPIO et timestamp avec `clock_gettime()` comme à l'étape 1) et les mêmes conditions (avec et sans un environnement surchargé `stress`) pour collecter les mêmes statistiques.
- ✓ Comparez cette nouvelle solution en termes de données statistiques avec la solution précédente.

Étape 4: Threads EVL

Nous allons maintenant exploiter des tâches EVL via la librairie `libevl`. Ecrivez une nouvelle version de votre tâche périodique dans le fichier `./rt_app/evl_timer.c` fourni. Ensuite, effectuez les mêmes tests de l'étape 3 avec `stress` et comparez vos résultats.

La documentation de l'API `libevl` est disponible [ici](#).

Quelques points pour la mise au point de votre premier programme Xenomai EVL:

1. Vous aurez besoin des fichiers d'entête suivants:

```
#include <evl/evl.h>
#include <evl/timer.h>
```

2. Au début de votre programme, utilisez la fonction `mlockall (MCL_CURRENT|MCL_FUTURE)`. Il s'agit d'une fonction offerte par Linux. Trouvez en quoi elle sera utile dans un contexte temps-réel.
3. Les tâches EVL démarrent comme une tâche classique, à l'aide d'un appel à la fonction `pthread_create`. Cela dit, vous êtes également libres d'utiliser le thread principal pour cet exercice mais ça ne sera pas toujours possible alors nous vous conseillons d'utiliser un nouveau thread
4. Le CPU 1 a été isolé du reste du système, comme observé précédemment lors du labo 01. Afin d'obtenir les meilleures performances en temps réel, il est nécessaire d'exécuter la tâche EVL sur ce CPU 1 isolé.
5. Afin de lier le thread que vous souhaitez au scheduler temps-réel, utilisez la fonction `evl_attach_self`
6. Afin de rendre la tâche courante périodique, nous aurons besoin d'un timer. La `libevl` permet d'en instancier un à l'aide de la fonction `evl_new_timer` (Utilisez la clock `EVL_CLOCK_MONOTONIC`)
7. La fonction `evl_read_clock` vous permet de récupérer le temps courant
8. Configurez les variables nécessaires afin d'avoir un timer se répétant à l'intervalle désiré (prévoyez un peu de marge de manoeuvre pour le démarrage) puis activez le timer à l'aide de la fonction `evl_set_timer`
9. La fonction `oob_read` permet d'attendre le timer
10. Terminez l'implémentation du programme en vous assurant de toujours utiliser des fonctions adaptée au contexte temps-réel. Par exemple, pour l'affichage, l'usage de la fonction `printf()` pose problème dans un contexte temps réel. Pour éviter les problèmes relatifs, utilisez la fonction `evl_printf()`, avec le même prototype.

Lorsque vous aurez écrit votre programme, utilisez `cmake` pour le compiler comme d'habitude



Commentez votre code et les résultats obtenus.

Travail à effectuer

Un rapport au format PDF devra contenir les réponses aux questions directes (indiquées par le symbole ✓)

- Rendez le rapport et le code sur cyberlearn, dans un fichier compressé nommé `rendu.tar.gz`
 - Ce fichier doit être généré en lançant le script `ptr_rendu.sh`

- Le script vérifie qu'un fichier `rapport.pdf` est présent à son niveau, ainsi qu'un dossier `code` également présent au même niveau

Les connaissances acquises durant les laboratoires seront également testées lors des tests écrits ainsi que lors de l'examen final.